

Deliverable 7: Security & Risk Assessment

What can go wrong, and what was deliberately left unsolved

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

Security & Risk Assessment

I'm not trying to solve every risk below. The assignment is explicit that spotting them unprompted matters more than closing every gap in a five-day prototype. What follows is what I actually thought about while building this, including a couple of things I only caught because I went looking, not because they were obvious.

Prompt injection

The single-stand copilot's prompt is built entirely from numbers I compute (anomaly score, threshold, per-signal deviations). There's no free-text field there, so this was mostly luck of the use case, not a designed defense.

That stopped being hypothetical the moment I added the fleet wide "ask about the fleet" free text box (`app/fleet_tools.py` , described in `docs/architecture.md` 's addendum). A supervisor can type anything into that box, so I actually had to design for it instead of getting it for free:

- **Scope refusal.** The system prompt tells the model its only job is answering questions about these 6 stands' sensor data, and to decline briefly and redirect if asked something unrelated. Tested this directly: asking it to write a poem or answer a general knowledge question, it declines and says what it can actually help with, instead of quietly turning into a general purpose chatbot riding on this feature's API key and system prompt.
- **Read only tools, always.** The five functions the model can call (current reading, all current readings, last alert, alert frequency, alert cause) only ever read computed data. None of them can change fleet state, trigger the simulator, or take any action. Worst case for a malicious or just confused question is a wrong sentence, never a side effect.
- **Tool output is data, not instructions.** The system prompt explicitly tells the model to treat everything a tool returns as data to report, never as commands to obey. Every current tool only returns numbers and timestamps, so there's nowhere for an instruction to actually hide today, but the boundary is stated now, before a future tool ever returns anything closer to free text (an operator's note field, say), the same boundary I said this doc would need if that day came, and now it's actually written into the running prompt, not just this document.
- **Bounded, not open ended.** The tool use loop is capped at 4 rounds so a confused model can't spin forever burning API calls on one question.

If a future version lets this ingest real free text from elsewhere (operator notes, maintenance logs), the same rule from the original version of this section still applies: that input is untrusted data passed to the model, never instructions, the same boundary I'd apply to any tool using agent.

Hallucination in decision support

This is the risk I actually designed around most deliberately, because it's the one most likely to matter in practice. The prompt explicitly tells the model not to invent sensor readings, timestamps, or history it wasn't given, and I saw the value of that constraint directly: even with it, the model's *interpretation* of a correctly-reported number was wrong once (the rolling-mean baseline bug in `docs/ai-partnership-log.md` , where it said "coolant pressure is up" about a number that was actually crashing). If a tightly-grounded prompt with a real number still produced a misleading sentence, an ungrounded one absolutely will. The mitigation isn't "trust the model less" in the abstract. It's that the raw numbers are always shown alongside the explanation on the dashboard, never replaced by it, so a supervisor isn't dependent on the sentence being right.

Data leakage

Every copilot call sends sensor readings and derived anomaly scores to Anthropic's API. That's data leaving Nucor's network by design, and it's worth naming plainly rather than treating an external LLM call as free. In this prototype the data is synthetic, so there's nothing to actually leak. In production, sensor telemetry off a specific stand is operationally sensitive (it tells you a lot about a plant's real capacity and reliability), and sending it to a third-party API needs an actual data processing agreement and a real answer to "does this count as sensitive operational data under whatever Nucor's data classification policy is," not an assumption that it's fine because it's "just numbers." The credential itself is the more mundane version of the same risk: it's kept in a gitignored local `.env`, never in source control, but a real deployment needs actual secrets management (a vault, not a file on someone's machine) and a plan for what happens if that key leaks.

Access control

Not implemented in the prototype. There's no login, no role separation between "can view" and "can act." That's a real gap, not an oversight I'm glossing over. In production, a shift supervisor viewing an alert and someone with authority to actually schedule a shutdown are not necessarily the same access level, and the tool doesn't currently distinguish them. This needs to be solved before this touches anything with real operational consequence, not as a follow-up.

Audit trail

Right now, an anomaly score and an alert either happened or they didn't, but there's no persistent log of who saw which alert, when, or what they did about it. For a tool that's supposed to influence real maintenance decisions, that's a gap. If a bearing does fail after the tool flagged it, "did anyone see this, and when" needs to be answerable from a log, not from memory. Straightforward to add (every alert render and every copilot call already has the data needed, it's just not being written anywhere durable) but it's not built.

Model deprecation and drift

Two different risks under one heading. First, the anomaly detector will drift. "Normal" operation changes over time (new equipment, process changes, seasonal effects), and a model trained once and never revisited will quietly get worse, either missing real failures or false-alarming more, without anyone noticing until it's bad enough to be obvious. There's no retraining cadence or drift monitoring in this prototype. Second, the copilot depends on a specific hosted model (Claude Haiku 4.5) that Anthropic could deprecate or change behavior on. The fallback path means the dashboard doesn't hard-break if that happens, but the *quality* of explanations would silently degrade to the rule-based version, and nothing currently alerts anyone that this happened. Both are acceptable-for-a-prototype gaps that would not be acceptable for anything running unattended.

The risk that worries me most, that isn't on the standard checklist

Alert fatigue. It's not a security risk in the traditional sense, but it's the one most likely to actually cause harm: even a false-alarm rate under 1% (1.06% with the original tuning, 0.05% after later fixes, see `docs/ai-partnership-log.md`) sounds fine in isolation, but scaled across every stand, every mill, every shift, continuously, it adds up into the kind of noise that gets a tool ignored within weeks. A tool that's ignored is worse than no tool at all, because it creates false confidence that someone's watching. I don't have a complete solution to this in the prototype (per-stand threshold

calibration and alert suppression logic would both help, and neither is built), but I think it's a more realistic failure mode for this project than most of the more textbook items above, and I'd rather name it plainly than leave it implicit.

What I deliberately did NOT build, as a risk-reduction decision

The system explains and suggests. It never triggers a shutdown, pages anyone automatically, or takes any action with real-world consequence on its own. That's covered in `docs/architecture.md`'s scope section, but it belongs here too: keeping a human in the loop for every consequential action is the single biggest risk mitigation in this whole design, and it was a decision I made on purpose, not a limitation I ran out of time to fix.